

CURSO DE MODELOS DE APRENDIZAJE · GOOGLE COLAB

Guía del docente

Clases 0 y 1

Cómo entender la clase y cómo explicarla (guion paso a paso)

Para el docente · úsala junto con los notebooks 00 y 01

Pablo Andrés Carvajal — Desarrollador Full Stack

pablocarvajal.dev

Para qué sirve esta guía

Este documento es **solo para ti (el docente)**. No es para proyectar. Te explica **cada concepto con palabras sencillas**, te da un **guion** de qué decir y qué mostrar, las **preguntas frecuentes** de los estudiantes con su respuesta, y los **errores comunes**. La idea: que puedas entender el tema aunque no seas experto y dictarlo con seguridad.

Tres materiales por clase y cómo se usan juntos:

Material	Quién lo ve	Cuándo
Presentación (presentaciones/)	Se proyecta a todos	Para introducir cada tema antes de tocar código
Notebook (notebooks/)	Tú lo ejecutas y ellos siguen	La parte práctica: correr celdas en vivo
Esta guía	Solo tú	Para prepararte y consultar mientras enseñas

EL RITMO IDEAL

Explica un concepto con **1–2 diapositivas** → cámbiate al **notebook** y ejecuta esa parte en vivo → deja que **ellos la ejecuten** → siguiente concepto. Nunca más de 5 minutos hablando sin tocar código.

Panorama: qué veremos y dónde encaja la IA

Esto es lo primero que conviene contarle a la clase el primer día: **a dónde vamos** y **qué tan lejos llega hoy la inteligencia artificial**.

¿Qué vamos a ver en todo el curso?

El curso son 6 notebooks que van de lo básico a lo avanzado. Las Clases 0 y 1 son la base:

#	Notebook	En una frase
0	Introducción y setup	Qué es ML, Colab y explorar datos (<i>esta clase</i>)
1	Supervisado básico	Predecir números y categorías, y medir el modelo (<i>esta clase</i>)
2	Supervisado avanzado	Pipelines, modelos potentes, validación y ajuste
3	No supervisado	Agrupar (clustering) y reducir dimensiones sin etiquetas
4	Mixto (semi-supervisado)	Aprender con pocas etiquetas
5	Proyecto final	Juntar todo en un proyecto propio

CÓMO LO EXPLICAS

"Hoy sembramos la base: entender qué es aprender de datos y hacer nuestros primeros modelos. Clase a clase vamos subiendo el nivel hasta un proyecto completo."

Los niveles de la IA (y en cuál estamos)

Hay dos formas útiles de "ordenar" la IA. La primera es como **muñecas rusas** (una dentro de otra):

```
INTELIGENCIA ARTIFICIAL (que una máquina imite comportamiento "inteligente")
└─ MACHINE LEARNING (aprende de datos, sin reglas escritas a mano) ← AQUÍ estamos
    └─ DEEP LEARNING (ML con redes neuronales; potencia a los LLM como ChatGPT)
```

Todo lo del curso vive en el nivel **Machine Learning**.

La segunda forma ordena la IA por **qué tan capaz es**:

Nivel	Qué es	¿Existe hoy?
IA estrecha (ANI)	Experta en UNA tarea concreta (recomendar videos, reconocer voz, traducir, jugar ajedrez, ChatGPT).	Sí — es toda la IA que existe hoy
IA general (AGI)	Igualaría a un humano en cualquier tarea intelectual.	No, todavía es una meta
Superinteligencia (ASI)	Superaría al humano en todo .	No, es hipotética

EL PUNTO CLAVE PARA LA CLASE

Estamos en la IA estrecha (ANI). Cada modelo que construiremos (predecir una propina, clasificar una flor, agrupar clientes) resuelve **una sola tarea**. No "piensa" ni "entiende": encuentra patrones en datos.

PREGUNTA QUE SEGURO TE HARÁN

"¿Y ChatGPT no es una IA general?" → No. Es IA **estrecha muy avanzada**: es buenísima en tareas de lenguaje, pero no tiene consciencia ni entiende como un humano. Sigue siendo ANI.

✓ Antes de la clase (10 min de preparación)

- ☐ Abre **Google Colab** (colab.research.google.com) con tu cuenta de Google.
- ☐ Sube el notebook de la clase (**Archivo** → **Subir notebook**) y **ejecútalo completo una vez** tú solo, para que no haya sorpresas.
- ☐ Ten la **presentación** abierta en el navegador (doble clic al archivo `.html`).
- ☐ Ten **esta guía** a un lado (impresa o en otra pantalla).
- ☐ Comprueba el internet del salón (Colab necesita conexión).

CONSEJO DE ORO

Ejecuta tú el notebook antes de la clase. Es la mejor forma de que te sientas seguro: si ya lo viste correr una vez, sabrás exactamente qué va a pasar en cada celda.



Parte 1 — Lo que TÚ debes dominar

Antes del guion, aquí tienes los conceptos explicados *para ti*, con la analogía que puedes reutilizar en clase. Si entiendes bien estos 6 puntos, dominas las clases 0 y 1.

1. ¿Qué es Machine Learning (ML)?

Idea central: en la programación normal, tú escribes las reglas. En ML, la máquina **descubre las reglas sola** mirando muchos ejemplos.

- **Programación tradicional:** reglas + datos → resultados. (Tú dices "si el correo dice GRATIS, es spam".)
- **Machine Learning:** datos + resultados → reglas (el "modelo"). (Le muestras 10.000 correos ya marcados y él descubre qué es spam.)

CÓMO LO EXPLICAS

"Imaginen que quiero enseñarle a un niño qué es un gato. No le doy una definición; le muestro muchas fotos de gatos. Con el tiempo, aprende solo. El Machine Learning es igual: aprende de ejemplos."

2. Los 3 tipos de aprendizaje

Tipo	¿Los datos traen la respuesta?	Para qué	Ejemplo
Supervisado	Sí (etiquetas)	Predecir	Predecir el precio de una casa
No supervisado	No	Descubrir grupos	Agrupar clientes parecidos
Mixto (semi-supervisado)	Pocas	Aprovechar pocas etiquetas	Clasificar con pocos datos etiquetados

En la **Clase 1** nos centramos en el **supervisado**, que es el más común y el más fácil de entender.

3. Regresión vs. Clasificación (dentro del supervisado)

- **Regresión** = predecir un **número** (un precio, una temperatura, una propina).
- **Clasificación** = predecir una **categoría** (spam/no spam, especie de flor).

TRUCO PARA RECORDARLO

Si la respuesta se puede poner en una regla ("**¿cuánto?**") es regresión. Si la respuesta es una etiqueta ("**¿cuál?**") es clasificación.

4. Entrenamiento y prueba (train / test) — ¡el concepto más importante!

Dividimos los datos en dos partes: con una el modelo **aprende** (train, ~80%) y con la otra lo **examinamos** (test, ~20%). Nunca lo evaluamos con los mismos datos que ya vio.

LA ANALOGÍA QUE SIEMPRE FUNCIONA

"Sería trampa evaluar a un estudiante con el mismo examen que ya tenía las respuestas. Por eso guardamos datos que el modelo nunca vio: para saber si **de verdad aprendió** o solo **memorizó**."

5. ¿Cómo sabemos si el modelo es bueno? (métricas)

Métrica	Para	Qué significa (en simple)
MAE	Regresión	En promedio, ¿por cuánto se equivoca? (ej.: "por \$0.90 de propina")
R²	Regresión	Qué tan bien explica los datos: 1 = perfecto, 0 = malo
Exactitud (accuracy)	Clasificación	% de aciertos
Matriz de confusión	Clasificación	Tabla que muestra en qué acierta y en qué se confunde

6. Overfitting (sobreajuste)

Un modelo **demasiado complejo** memoriza los datos de entrenamiento y falla con datos nuevos. Le va perfecto en el "examen que ya vio" y mal en preguntas nuevas.

CÓMO LO EXPLICAS

"Es como el estudiante que memoriza las respuestas de memoria pero no entiende el tema: si le cambias la pregunta, falla. Queremos modelos que **entiendan**, no que **memoricen**."

● Parte 2 — Guion de la CLASE 0 (Introducción y setup)

Meta de la clase: que todos entiendan qué es ML, se sientan cómodos en Colab y exploren su primer dataset. Notebook: `00_introduccion_y_setup.ipynb`.

Duración sugerida: 1.5–2 horas (con calma, es la primera toma de contacto).

Apertura (5 min) — qué decir

"Hoy no vamos a memorizar teoría: vamos a **usar** inteligencia artificial escribiendo unas pocas líneas y viendo resultados al instante. No necesitan instalar nada, todo corre en la nube de Google."

Sección por sección

① ¿Qué es Machine Learning? (diapositivas + hablar)

Usa la analogía del niño y los gatos (punto 1 de la Parte 1). Presenta los **3 tipos** con la tabla. *No profundices*: solo que capten la idea. Diles que hoy es la base y que la práctica fuerte empieza en la Clase 1.

② Google Colab en 1 minuto (mostrar en pantalla)

Abre Colab y muestra en vivo:

- Qué es una **celda de código** y una de **texto**.
- Cómo se ejecuta: **Shift + Enter** (hazlo con la primera celda: `print("¡Hola!...")`).
- Que el número `[1]` a la izquierda indica el orden en que se ejecutó.

PÍDELES QUE LO HAGAN

Que cada uno abra el notebook y ejecute la primera celda. No sigas hasta que todos vean el mensaje. Este momento vence el miedo inicial.

③ Las librerías (celda de imports)

Explica el *alias*: "escribimos `import pandas as pd` para no repetir 'pandas' todo el tiempo; `pd` es su apodo". Menciona qué hace cada una en una frase (pandas = tablas, matplotlib/seaborn = gráficas, scikit-learn = los modelos).

④ Primer dataset: Iris (celdas de carga y exploración)

Ejecuta la carga y muestra `df.head()`. Explica que cada fila es una flor y cada columna una medida. Luego `df.describe()` y `value_counts()`: "así miramos los datos antes de modelar; siempre se empieza mirando".

⑤ Primera gráfica (scatter de pétalos)

Ejecuta el `scatterplot`. Pregunta a la clase: "*¿ven cómo los colores forman grupos separados?*".

Remata: "por eso una máquina podrá aprender a diferenciarlas; eso lo haremos en la próxima clase".

⑥ Ejercicio + cierre

Déjalos intentar el ejercicio del notebook (últimas filas, promedio por especie, otro scatter). Cierra recordando los 3 tipos de aprendizaje y anuncia la Clase 1.

PREGUNTAS FRECUENTES — CLASE 0

"**¿Necesito saber matemáticas?**" → No para este curso; usaremos librerías que hacen las cuentas. · "**¿Se me borra el trabajo?**" → Colab guarda en tu Google Drive si eliges "Guardar una copia en Drive". · "**¿Por qué sale un número entre corchetes?**" → Es el orden de ejecución de la celda. · "**¿Y si una celda da error?**" → Casi siempre es porque no ejecutaron una celda anterior; ejecutar desde arriba lo arregla.

Parte 3 — Guion de la CLASE 1 (Aprendizaje supervisado básico)

Meta: entrenar sus primeros modelos (regresión y clasificación), evaluarlos y entender el overfitting.
Notebook: `01_supervisado_basico.ipynb` . **Duración:** 3–4 horas.

Apertura (5 min)

"En la Clase 0 miramos datos. Hoy vamos a **predecir**: primero un número (cuánta propina) y luego una categoría (qué especie de flor). Y aprenderemos a medir si nuestro modelo es bueno."

PARTE A — Regresión: predecir un número

① El problema (propinas)

Muestra el dataset `tips` y el scatter propina vs cuenta. Pregunta: "*¿la propina sube con la cuenta?*" Sí → "un modelo puede aprender esa relación con una línea".

② **X, y, y train/test**

Explica: **X** es lo que entra (la cuenta), **y** es lo que queremos predecir (la propina). Aplica aquí la analogía del examen (punto 4 de la Parte 1). Es EL momento clave de la clase, tómate tu tiempo.

③ Entrenar y ver la fórmula

Ejecuta `LinearRegression().fit(...)` . Muestra la fórmula aprendida (pendiente e intercepto): "el modelo encontró la mejor línea; por cada dólar de cuenta, la propina sube tanto".

④ Evaluar (MAE y R^2) y graficar la línea

Interpreta el **MAE** en dólares ("en promedio nos equivocamos por ~X") y el **R^2** ("explica el Y% de la variación"). Muestra la línea roja sobre los puntos: es lo que el modelo "aprendió".

PARTE B — Clasificación: predecir una categoría

⑤ kNN (vecinos)

Analogía: "para saber la especie de una flor nueva, miramos las 5 flores más parecidas y votamos". Ejecuta y muestra la exactitud.

⑥ Árbol de decisión

Analogía: "es como un juego de preguntas sí/no: ¿el pétalo mide más de X? → por aquí; ¿y de Y? → por allá". Ejecuta y compara la exactitud con kNN.

⑦ Matriz de confusión

Ejecuta la matriz. Explícala: "la diagonal son los aciertos; fuera de la diagonal, las confusiones". Señala una celda concreta: "aquí confundió 1 flor de esta especie con esta otra".

PARTE C — Overfitting

⑧ La gráfica train vs test

Ejecuta la celda que varía la profundidad del árbol. Señala: "miren cómo el entrenamiento sube hacia 100%, pero la prueba se estanca o baja: eso es **overfitting**". Usa la analogía del estudiante que memoriza (punto 6).

⑨ Ejercicios + cierre

Déjalos con los ejercicios (cambiar el número de vecinos, probar otro modelo, un árbol de profundidad 1). Cierra: "ya saben entrenar, evaluar y detectar overfitting; en la Clase 2 haremos modelos más potentes".

PREGUNTAS FRECUENTES — CLASE 1

"¿Por qué el resultado da un poco distinto cada vez?" → Por el azar en la división de datos; por eso usamos `random_state=42`, para que sea reproducible. · "¿Cuál modelo es mejor, kNN o árbol?" → Depende de los datos; por eso probamos varios y comparamos. · "¿100% de exactitud no es sospechoso?" → En Iris es normal porque es un problema fácil y pequeño; en datos reales casi nunca pasa. · "¿Qué es un buen R^2 ?" → Depende del problema; más cerca de 1 es mejor, pero 0.5 puede ser útil según el caso.



Parte 4 — Entendiendo el código, celda por celda

Aquí explicamos **qué hace cada celda de código** de los dos notebooks, en lenguaje simple, para que puedas **leer el código, entenderlo y explicarlo**. No necesitas memorizar nada: con captar la *idea* de cada línea es suficiente.

4 PALABRAS QUE SE REPITEN EN TODO SCIKIT-LEARN

`import` = traer una herramienta para poder usarla · `.fit(X, y)` = *entrenar* (el modelo aprende de los datos) · `.predict(X)` = *predecir* con el modelo ya entrenado · `.score(...)` = *calificar* qué tan bien lo hace. Cuando entiendes estas 4, entiendes casi todo el curso.

■ Notebook 0 — celda por celda

Celda 1 — Comprobar que Python funciona

```
print("¡Hola! Colab está funcionando ✅")  
print("2 + 2 =", 2 + 2)
```

- `print(...)` **muestra en pantalla** lo que le pongas entre paréntesis.
- La segunda línea muestra un texto y, separado por una coma, el **resultado de una operación** (4).
- **Objetivo real:** que el estudiante ejecute su primera celda y pierda el miedo.

Celda 2 — Importar las librerías

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
import sklearn  
sns.set_theme()          # estilo bonito para las gráficas  
print(np.__version__)    # ... imprime las versiones
```

- `import pandas as pd` = "trae pandas y de ahora en adelante lo llamo `pd`" (un apodo/alias).
- Cada librería tiene su trabajo: **numpy** (números), **pandas** (tablas), **matplotlib/seaborn** (gráficas), **sklearn** (los modelos).
- `sns.set_theme()` solo hace que las gráficas se vean mejor.
- `np.__version__` muestra el número de versión (solo para confirmar que todo cargó).

Celda 3 — Cargar el dataset Iris

```
from sklearn.datasets import load_iris  
iris = load_iris(as_frame=True)  
df = iris.frame  
df["especie"] = iris.target_names[iris.target]  
df.head()
```

- `load_iris(as_frame=True)` trae el dataset ya como **tabla de pandas**.
- `df = iris.frame` guarda esa tabla en una variable llamada `df` (de *dataframe*, "tabla").
- La línea de `especie` **crea una columna nueva** con el nombre de la flor (setosa/versicolor/virginica) en vez de un número 0/1/2, para que se lea fácil.
- `df.head()` muestra las **primeras 5 filas** para echar un vistazo.

QUÉ DECIR

"Cada fila es una flor; cada columna, una medida. La última columna es la respuesta: su especie."

Celda 4 — Explorar la tabla

```
print("Forma:", df.shape)
print(df["especie"].value_counts())
df.describe()
```

- `df.shape` dice (**cuántas filas, cuántas columnas**) → (150, 6).
- `df["especie"].value_counts()` **cuenta cuántas hay de cada especie** (50 y 50 y 50).
- `df.describe()` da **estadísticas** de cada columna numérica: promedio, mínimo, máximo, etc.
- **Mensaje:** siempre miramos los datos *antes* de modelar.

Celda 5 — Primera gráfica (scatter)

```
sns.scatterplot(data=df, x="petal length (cm)", y="petal width (cm)", hue="especie")
plt.show()
```

- `scatterplot` dibuja un **punto por flor**: posición según largo (x) y ancho (y) del pétalo.
- `hue="especie"` pinta cada especie de un **color distinto**.
- `plt.show()` muestra la gráfica.
- **Lo importante:** se ven **3 grupos separados** → por eso un modelo podrá distinguirlas (eso es la Clase 1).

Celda 6 — Bonus: pairplot

```
sns.pairplot(df, hue="especie")
plt.show()
```

- Dibuja **todas las combinaciones** de medidas a la vez, en una cuadrícula de gráficas.
- Sirve para ver rápido qué medidas separan mejor las especies. (La 7ª celda es el ejercicio, va vacía.)

■ Notebook 1 — celda por celda

Este notebook tiene dos mitades: **Parte A (regresión)** y **Parte B (clasificación)**, más una **Parte C (overfitting)**. El patrón siempre es el mismo: *separar datos* → *dividir train/test* → *entrenar* (*fit*) → *evaluar*.

Celda 1 — Importar librerías

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
sns.set_theme()
```

- Lo mismo que en el Notebook 0: trae las herramientas que usaremos.

Celdas 2–3 — Cargar propinas y graficarlas (Parte A)

```
tips = sns.load_dataset("tips")
tips.head()
# ...
sns.scatterplot(data=tips, x="total_bill", y="tip")
plt.show()
```

- `load_dataset("tips")` descarga una tabla de propinas de restaurante (necesita internet, Colab lo tiene).
- El scatter muestra **total de la cuenta (x)** vs **propina (y)**. Se ve que suben juntas.
- **Qué preguntar:** "¿la propina crece con la cuenta?" → sí → una línea puede describir esa relación.

Celda 4 — Separar X e y

```
X = tips[["total_bill"]] # lo que ENTRA (dobles corchetes = tabla)
y = tips["tip"]          # lo que queremos PREDECIR
```

- **X** = las *features* (entradas). Aquí, solo el total de la cuenta.
- **y** = el *objetivo* (la respuesta). Aquí, la propina.
- El detalle de los **dobles corchetes** en X es porque scikit-learn espera una tabla (2D) como entrada.

Celda 5 — Dividir en entrenamiento y prueba

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `train_test_split` parte los datos en dos: **80% para aprender** y **20% para examinar**.
- `test_size=0.2` = 20% para prueba. `random_state=42` = fija el azar para que siempre salga igual (reproducible).
- Devuelve **4 piezas**: entradas y respuestas, de train y de test.
- **Este es EL concepto clave** (usa la analogía del examen).

Celda 6 — Entrenar la regresión lineal

```
from sklearn.linear_model import LinearRegression
modelo = LinearRegression()
modelo.fit(X_train, y_train)    # AQUÍ aprende la línea
```

- `LinearRegression()` crea el modelo (aún "vacío", no sabe nada).
- `.fit(X_train, y_train)` = **entrenar**: encuentra la mejor línea que relaciona cuenta → propina.
- Luego imprime la **fórmula aprendida** (una pendiente y un punto de partida): "por cada dólar de cuenta, la propina sube tanto".

Celda 7 — Evaluar (MAE y R²)

```
from sklearn.metrics import mean_absolute_error, r2_score
pred = modelo.predict(X_test)
mean_absolute_error(y_test, pred)    # error promedio en dólares
r2_score(y_test, pred)               # qué tan bien explica (0 a 1)
```

- `.predict(X_test)` = el modelo **predice** la propina de las cuentas que *no* vio.
- `mean_absolute_error` compara predicción vs realidad → "en promedio nos equivocamos por \$X".
- `r2_score` → cercano a 1 es bueno.

Celdas 8–9 — Dibujar la línea y predecir un caso nuevo

```
plt.plot(x_linea, modelo.predict(x_linea), color="red")    # la línea aprendida
# ...
cuenta = pd.DataFrame({"total_bill": [50]})
modelo.predict(cuenta)    # propina estimada para una cuenta de $50
```

- La primera dibuja en **rojo** lo que el modelo aprendió, encima de los puntos reales.

- La segunda **usa el modelo** con un dato inventado (\$50) para ver su predicción. Así se usa un modelo en la vida real.

Celda 10 — Empieza la Parte B: cargar Iris y dividir

```
iris = load_iris(as_frame=True)
X = iris.data      # las 4 medidas
y = iris.target    # 0,1,2 = especie
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- Ahora **X son las 4 medidas** de la flor e **y es la especie** (como número).
- Mismo `train_test_split` de antes: el patrón se repite.

Celda 11 — Modelo 1: kNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
accuracy_score(y_test, knn.predict(X_test)) # % de aciertos
```

- **kNN** clasifica una flor nueva mirando sus **5 vecinas más parecidas** y votando.
- Mismo trío de siempre: crear → `fit` (entrenar) → `predict` + `accuracy_score` (evaluar).
- `accuracy_score` = **exactitud**: porcentaje de flores del test que acertó.

Celda 12 — Modelo 2: árbol de decisión

```
from sklearn.tree import DecisionTreeClassifier
arbol = DecisionTreeClassifier(max_depth=3, random_state=42)
arbol.fit(X_train, y_train)
accuracy_score(y_test, arbol.predict(X_test))
```

- El **árbol** aprende reglas sí/no ("¿el pétalo mide más de X?"). `max_depth=3` limita cuántas preguntas encadena (para no memorizar).
- Se entrena y evalúa igual que el kNN, y se comparan las exactitudes.

Celda 13 — Matriz de confusión

```
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay
cm = confusion_matrix(y_test, pred)
ConfusionMatrixDisplay(cm, display_labels=nombres).plot()
classification_report(y_test, pred, target_names=nombres)
```

- La **matriz** es una tabla: filas = especie real, columnas = lo que predijo. La **diagonal son los aciertos**.
- `classification_report` da precision/recall/F1 por especie (métricas más finas; con mencionarlas basta en este nivel).

Celda 14 — Predecir una flor nueva

```
flor = pd.DataFrame([[5.1, 3.5, 1.4, 0.2]], columns=iris.feature_names)
nombres[arbol.predict(flor)[0]] # devuelve el nombre de la especie
```

- Creamos una flor con 4 medidas inventadas y el árbol dice qué especie es. Igual que el caso de la propina, pero con categorías.

Celda 15 — Overfitting (Parte C)

```
for d in range(1, 11):
    m = DecisionTreeClassifier(max_depth=d, random_state=42).fit(X_train, y_train)
    train_scores.append(m.score(X_train, y_train))
    test_scores.append(m.score(X_test, y_test))
# ... luego grafica train vs test
```

- Es un **bucle** (`for`): entrena un árbol con profundidad 1, 2, 3... hasta 10.
- `m.score(...)` = exactitud; guardamos la de **train** y la de **test** en cada vuelta.
- La gráfica muestra que el train sube a 100% pero el test se estanca → **overfitting** (memoriza, no aprende).
- (La última celda es el ejercicio, va vacía.)

LA GRAN CONCLUSIÓN

Todos los modelos de scikit-learn se usan igual: **crear** → **fit** → **predict** → **evaluar**. Cambia el modelo (kNN, árbol, regresión...), pero **el patrón es siempre el mismo**. Eso es lo que tus estudiantes deben interiorizar.



Errores comunes en Colab y cómo resolverlos

Síntoma	Causa probable	Solución
<code>NameError: 'X' is not defined</code>	No ejecutaron una celda anterior	Ejecutar desde arriba (Entorno de ejecución → Ejecutar todas)
La gráfica no aparece	Falta ejecutar la celda o falta <code>plt.show()</code>	Volver a ejecutar la celda
Un dataset (tips/penguins) no carga	Sin internet	Revisar conexión; reintentar la celda
"Sesión caducada" o se reinició	Colab cerró el entorno por inactividad	Reconectar y Ejecutar todas de nuevo (los datos se recargan solos)
Cambié algo y ahora da error raro	Estado desordenado de celdas	Entorno de ejecución → Reiniciar y ejecutar todo

TU COMODÍN UNIVERSAL

Si algo se rompe en clase y no sabes por qué: **Entorno de ejecución → Reiniciar y ejecutar todo**. Resuelve el 90% de los problemas.



Glosario rápido

Término	En una frase
Dataset	La tabla de datos con la que trabajamos.
Feature	Una columna de entrada (una característica). Se guardan en <code>x</code> .
Etiqueta / target	La respuesta que queremos predecir. Se guarda en <code>y</code> .
Modelo	La "fórmula" que el algoritmo aprende de los datos.
Entrenar (fit)	El proceso en que el modelo aprende de los datos de train.
Predecir (predict)	Usar el modelo ya entrenado sobre datos nuevos.
train / test	Datos para aprender / datos para examinar.
Overfitting	Cuando el modelo memoriza en vez de aprender.
random_state	Una "semilla" para que el azar dé siempre el mismo resultado (reproducibilidad).

MENTALIDAD PARA ENSEÑAR

No tienes que saberlo todo. Si un estudiante pregunta algo que no sabes, di: **"Buena pregunta, vamos a averiguarlo"** y pruébenlo en el notebook. En ML, experimentar ES el método.